

NeuroAdapt

Autonomous Neuro-Diverse Learning Ecosystem

DEVELOPMENT WORKFLOW & TEAM TASK ALLOCATION

Prarthana Upadhyaya Frontend · Observer Layer · QML Integration	Sudarshan S. Niranjana Backend · Orchestrator · Quantum Core	Varun Aditya Generative Engine · Feedback Loop · DevOps
---	--	---

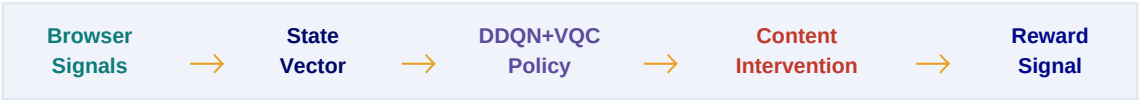
Colour Legend

Prarthana (P1)	Sudarshan (P2)	Varun (P3)	All Three
----------------	----------------	------------	-----------

1. Project Overview & Architecture

NeuroAdapt is a four-layer adaptive learning system. The diagram below shows how data flows through each layer every 30-second cycle:

Layer	Component	Core Technology	Primary Owner
1	The Observer (telemetry)	Vanilla JS · Browser Performance API LocalStorage (state only)	P1
2	The Orchestrator (policy net)	Python · PyTorch · PennyLane (VQC) Double DQN (DDQN)	P2
3	Generative Synthesis Engine	FastAPI · LLaMA-3 (Ollama) / OpenAI API Stable Diffusion (local) · Coqui TTS	P3
4	Feedback Loop & Data Pipeline	PostgreSQL · Redis · Next.js dashboard LTI 1.3 (future)	P3 + P1



2. Full Tech Stack

Domain	Technology	Purpose	Owner
Quantum ML	PennyLane 0.38+	VQC definition, RX/RN/CNOT gates, Z-basis measurement	P2
Quantum ML	PyTorch 2.x	DDQN training loop, replay buffer, optimiser	P2
Quantum ML	NumPy / SciPy	State vector normalisation, reward shaping	P2 + P3
Quantum Sim	PennyLane default.qubit	CPU simulator for POC (5-qubit VQC)	P2

Quantum Sim	IBM Qiskit (optional)	Future NISQ hardware migration path	P2
Backend	Python 3.11 + FastAPI	REST API, session management, inference endpoint	P2
Backend	PostgreSQL 16	Student session logs, reward history, Preference Delta	P2
Backend	Redis 7	Real-time state vector cache (sub-5 ms reads)	P2
Frontend	Next.js 14 (React)	Student UI, Energy Bar, lesson renderer, dashboard	P1
Frontend	Tailwind CSS	Accessible, distraction-reduced UI (WCAG 2.1 AA)	P1
Frontend	Browser Perf API + JS	Telemetry: dwell, jitter, tab-switch, stall, delta	P1
Gen Engine	LLaMA-3 via Ollama	Local LLM for text simplification & reading-level rewrite	P3
Gen Engine	OpenAI API (fallback)	Cloud LLM if local model unavailable	P3
Gen Engine	Stable Diffusion (local)	Visual content generation triggered by Orchestrator	P3
Gen Engine	Coqui TTS / ElevenLabs	Calm-preset audio; Coqui = free, ElevenLabs = premium	P3
Gen Engine	HeyGen API (optional)	Avatar video generation for video intervention	P3
Auth / Infra	JWT + HTTPS	Stateless auth; no PII leaves browser	P2
Auth / Infra	Docker + Docker Compose	Containerised full-stack for reproducible dev/deploy	P3
Auth / Infra	GitHub Actions CI/CD	Lint, unit test, build on every PR	P3
Eval	Weights & Biases (W&B)	Training curves, Preference Delta tracking, ablation logs	P2

3. Development Phases

Phase 0: Foundation & Setup

Week 1–2

All

- Agree on Git branching strategy: main / dev / feature-* / quantum-*
- Scaffold monorepo: /frontend /backend /quantum /gen-engine /infra
- Set up Docker Compose with Postgres + Redis + FastAPI skeleton + Next.js dev server
- Create shared constants file: STATE_VECTOR_DIMS=5, ACTION_SPACE=6, GAMMA=0.99, etc.

Phase 1: Observer Layer (Telemetry)

Week 2–4

P1 (lead)

- Implement JS telemetry module: window.addEventListener hooks for mousemove, blur/focus, scroll
- Compute Semantic Dwell Ratio: $(\text{timeOnSlide ms}) / (\text{wordCount} \times \text{averageReadingTime})$
- Compute Interaction Jitter: rolling std-dev of mouse velocity over last 10 samples
- Compute Focus Persistence: count of document.visibilityState==='hidden' events per 30 s window
- Compute Stall Duration: $\text{Date.now()} - \text{lastInteractionTimestamp}$
- Capture Preference Delta: record learner's end-of-lesson format modal selection
- Normalise all 5 signals to [0, 1] range client-side; package as JSON state vector
- POST normalised vector to /api/state every 30 s; no raw data ever transmitted

P2 (support)

- Define /api/state FastAPI endpoint; validate incoming state vector schema (Pydantic)
- Write Redis SET for latest vector keyed by session_id (TTL 5 min)

P3 (support)

- Build a mock data generator (Python script) that emits synthetic state vectors for offline DDQN training

Phase 2: Quantum-Classical Orchestrator

Week 3–6

P2 (lead)

- Implement VQC in PennyLane: 5 qubits (one per signal), RX(feature_i) encoding layer
- Add entanglement layer: CNOT(0→1), CNOT(1→2), CNOT(2→3), CNOT(3→4) — captures co-variance
- Add second rotation layer: RY(trainable_param_i) on each qubit
- Measure Z-basis expectation: `qml.expval(qml.PauliZ(i))` for `i` in `0..4` → 5 classical outputs
- Wrap VQC as a TorchLayer; connect to two fully-connected heads (advantage + value streams = Dueling DDQN)
- Implement replay buffer (deque, capacity 10,000), epsilon-greedy exploration (ϵ : 1.0→0.05)
- Implement Double DQN training loop: online net + target net, soft update every 100 steps
- Reward function: +1 lesson complete, +0.5 correct answer, −1 Energy Bar pressed, −0.3 tab-switch, +0.2 Pref Delta improvement
- Save model checkpoint after every 50 episodes; log loss + Q-values to W&B;

P1 (support)

- Build Preference Delta feedback widget in Next.js: end-of-lesson modal (Text / Video / Audio / Game)
- Integrate Energy Bar component: a progress-style element the student clicks to force a sensory break
- POST both signals to `/api/feedback` endpoint

P3 (support)

- Implement `/api/action` FastAPI endpoint that calls trained policy; returns one of 6 action IDs
- Persist (state, action, reward, next_state) tuples to Postgres for offline re-training

Phase 3: Generative Synthesis Engine**Week 4–7****P3 (lead)**

- Set up Ollama locally; pull llama3 model; write `text_simplify(text, level)` function
- Implement prompt template for each reading level (Grade 5 / Grade 8 / University) — use few-shot examples
- Integrate Stable Diffusion (diffusers library) for visual generation; wire to `action_id==3` (Switch to Video fallback: image)
- Integrate Coqui TTS: clone repo, load `tts_models/en/ljspeech/tacotron2-DDC`, generate WAV from simplified text
- Implement async pre-fetching: when Observer posts state, begin generating next-format variant in background
- Build gamified task injector: on `action_id==4`, return a multiple-choice quiz JSON with difficulty scaled to current level
- Implement sensory break screen: `action_id==5` → blank screen + timer countdown + soft TTS prompt

P2 (support)

- Add confidence threshold gate in Orchestrator: only trigger gen engine if max Q-value > 0.60
- Expose /api/generate endpoint; accept {action_id, slide_content, learner_level}

P1 (support)

- Build dynamic content renderer in Next.js: conditionally render text / image / audio / quiz based on response
- Implement content transition animations (CSS fade, 300 ms) — avoid flash/flicker for autism profiles

Phase 4: Integration & POC Testing

Week 7–9

P2 (lead)

- Wire full pipeline: Observer → Redis → Orchestrator → Gen Engine → Frontend render → Feedback → Reward → Retrain
- Run ablation A: remove Focus Persistence signal; measure change in Preference Delta convergence speed
- Run ablation B: reduce action space to 3 (hold, simplify, break); compare API cost vs engagement
- Run ablation C: replace VQC layer with classical dense layer (same param count); compare convergence epochs to 10% Pref Delta
- Log all ablation results to W&B; export comparison plot as PNG for submission report

P1 (lead)

- Recruit 20–30 student testers (RVCE batch); obtain informal consent
- Collect 3 × 20-min sessions per student across one subject module
- Compute engagement metric: (slides_completed / slides_total) per session; compare week 1 vs week 3
- Run accessibility audit: axe-core automated scan + manual keyboard navigation test

P3 (lead)

- Deploy full stack to a single VM (DigitalOcean / AWS EC2 free tier) using Docker Compose
- Implement basic rate-limiting on /api/generate to prevent cost overruns
- Set up monitoring: Prometheus + Grafana for API latency, Redis hit rate, queue depth
- Write seed data SQL script for demo: 3 pre-baked learner profiles (ADHD-like, dyslexia-like, neurotypical control)

Phase 5: Report, Demo & Submission

Week 9–10

All

- P1: Write frontend & UX sections of report; prepare UI walkthrough video (Loom, 5 min)
 - P2: Write quantum methodology section; include VQC circuit diagram and convergence graph
 - P3: Write generative engine section; include latency benchmarks for each gen modality
 - All: Compile final report using LaTeX / Overleaf; proof-read + add references from abstract
 - P2: Package model weights + training script in /quantum folder; add README with reproduce instructions
 - P3: Write deployment README; push Docker image to Docker Hub or GitHub Container Registry
 - All: Prepare 10-slide pitch deck for Unisys panel; P1 presents UI, P2 presents quantum core, P3 presents business/ROI
-

4. Quantum Network — Detailed Guide

This section provides step-by-step implementation guidance for the Variational Quantum Circuit (VQC) inside the DDQN Orchestrator. All three team members have quantum responsibilities; this guide covers the shared conceptual foundation before detailing individual tasks.

4.1 Why a VQC inside a DDQN?

A standard DDQN uses two fully-connected layers as its feature extractor. NeuroAdapt replaces the inner hidden layer with a Variational Quantum Circuit. The VQC maps the 5-dimensional state vector into a higher-dimensional Hilbert space ($2^5 = 32$ basis states for 5 qubits) using exponentially fewer parameters than a classical dense layer of equivalent width. The key advantage for NeuroAdapt is faster convergence in the first 20–40 episodes — exactly the window where a new learner's profile must be established.

4.2 VQC Architecture (implement in `pennylane_vqc.py`)

The circuit has three layers:

Layer	Operation	Purpose
Encoding	$RX(s_i)$ on qubit i for $i=0..4$	Embed each normalised signal as a rotation angle on the Bloch sphere
Entanglement	$CNOT(i \rightarrow i+1)$ for $i=0..3$	Couple adjacent signals — e.g. high Focus Persistence + high Stall = parallel
Variational	$RY(\theta_i)$ on qubit i (trainable)	Learned rotations; these are the parameters PyTorch will differentiate
Measurement	Z-basis expval on each qubit	Collapses each qubit to a scalar in $[-1, +1]$; gives 5 classical outputs to DDQN

4.3 Starter Code Skeleton (P2 implements; P1 & P3 review)

File: `quantum/pennylane_vqc.py`

```
import pennylane as qml
import torch
import torch.nn as nn

N_QUBITS = 5          # one per Observer signal
N_ACTIONS = 6         # DDQN output dimension
dev = qml.device("default.qubit", wires=N_QUBITS)

@qml.qnode(dev, interface="torch", diff_method="backprop")
def vqc(inputs, weights):
    # --- Encoding layer: embed state vector as rotation angles ---
    for i in range(N_QUBITS):
        qml.RX(inputs[i] * 3.14159, wires=i) # scale to [0, π]

    # --- Entanglement layer: nearest-neighbour CNOT chain ---
    for i in range(N_QUBITS - 1):
        qml.CNOT(wires=[i, i + 1])

    # --- Variational layer: trainable RY rotations ---
    for i in range(N_QUBITS):
        qml.RY(weights[i], wires=i)

    # --- Measurement: Z expectation per qubit ---
    return [qml.expval(qml.PauliZ(i)) for i in range(N_QUBITS)]
```

```

class QuantumDDQN(nn.Module):
    def __init__(self):
        super().__init__()
        weight_shapes = {"weights": (N_QUBITS,)}
        self.quantum_layer = qml.qnn.TorchLayer(vqc, weight_shapes)
        # Dueling streams (advantage + value)
        self.advantage = nn.Linear(N_QUBITS, N_ACTIONS)
        self.value      = nn.Linear(N_QUBITS, 1)

    def forward(self, state):
        q_out = self.quantum_layer(state)          # [batch, 5]
        adv   = self.advantage(q_out)              # [batch, 6]
        val   = self.value(q_out)                  # [batch, 1]
        return val + adv - adv.mean(dim=1, keepdim=True) # Dueling Q-values

```

4.4 Training Loop Checklist (P2)

- Initialise online_net and target_net (both QuantumDDQN); copy weights at start
- Replay buffer: collections.deque(maxlen=10_000); each entry = (s, a, r, s', done)
- Epsilon schedule: $\epsilon = \max(0.05, 1.0 - \text{episode}/500)$
- Sample mini-batch of 32; compute TD target: $r + \gamma \max_a Q_{\text{target}}(s', a)$
- Loss: $F_{\text{smooth_l1_loss}}(Q_{\text{online}}(s)[a], \text{target})$ — Huber loss is more stable than MSE for RL
- Soft update target net every 100 steps: $\theta_{\text{target}} \leftarrow 0.005 \cdot \theta_{\text{online}} + 0.995 \cdot \theta_{\text{target}}$
- Log episode reward, ϵ , Q-mean to W&B; every episode
- Save checkpoint (torch.save) every 50 episodes to /quantum/checkpoints/

4.5 Quantum Tasks by Member

P1 – Prarthana

- Build the W&B; dashboard view that plots Preference Delta per episode (frontend chart embedded in Next.js admin panel)
- Write a Python helper script (quantum/visualise_circuit.py) that calls qml.draw(vqc) and saves the circuit diagram as PNG for the submission report
- Implement the quantum confidence gate in the frontend: if the API returns confidence < 0.60, display a 'thinking...' skeleton loader instead of triggering a gen API call
- Review P2's VQC code: confirm RX encoding range, check entanglement topology matches the ADHD co-activation hypothesis in the abstract

P2 – Sudarshan

- Implement the full VQC + DDQN as described in §4.3; run unit tests with synthetic state vectors
- Perform ablation C: swap VQC layer for nn.Linear(5,5); train both versions on synthetic data for 300 episodes; export convergence plot
- Profile VQC inference time on CPU (default.qubit); ensure p50 latency < 200 ms per forward pass
- Document Qiskit migration path: write a short /quantum/QISKIT_MIGRATION.md noting which PennyLane device string to change and expected noise model differences
- Train on POC cohort data (Phase 4); present Preference Delta convergence graph showing VQC reaches $\Delta < 0.1$ in fewer episodes than classical baseline

- Write the reward engineering module (quantum/reward.py): implement the full reward function from §3 with configurable weights (YAML config file)
 - Integrate reward computation into /api/feedback endpoint: on each learner feedback POST, compute reward, push (s, a, r, s') to Postgres replay table
 - Build the offline re-training script (quantum/retrain.py): reads last N rows from replay table, runs 50 training epochs, saves new checkpoint; schedule via cron / GitHub Actions
 - Implement the mock-data generator (quantum/mock_data.py): generate 3 synthetic learner archetypes (ADHD hyperfocus, dyslexia slow-reader, neurotypical) as 500-episode datasets for pre-training before POC
-

5. 10-Week Timeline at a Glance

Task / Owner	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10
Repo & Docker setup (All)	■	■								
Observer telemetry JS (P1)		■	■	■						
FastAPI skeleton + Redis (P2)	■	■	■							
Mock data generator (P3)		■	■							
VQC + DDQN core (P2)			■	■	■	■				
Energy Bar + Pref Delta UI (P1)				■	■					
Gen Engine: LLM + TTS (P3)				■	■	■	■			
/api/action endpoint (P2+P3)					■	■				
Frontend content renderer (P1)					■	■	■			
Ablation experiments (P2)							■	■		
POC student testing (P1)							■	■	■	
VM deployment (P3)							■	■		
Report & deck (All)									■	■

6. Quick Reference — Files, Endpoints & Config

Key Files

File Path	Owner	Description
/frontend/components/Observer.js	P1	Telemetry module — all 5 signals + POST to /api/state
/frontend/components/EnergyBar.jsx	P1	Manual break override widget; fires /api/feedback
/frontend/components/ContentRenderer.jsx	P1	Switches between text / image / audio / quiz based on action
/backend/main.py	P2	FastAPI app entry; registers all routers
/backend/routers/state.py	P2	POST /api/state — validates + caches state vector
/backend/routers/action.py	P2	GET /api/action — calls policy net, returns action_id + confidence
/backend/routers/feedback.py	P3	POST /api/feedback — computes reward, writes to Postgres
/quantum/pennylane_vqc.py	P2	VQC circuit + QuantumDDQN class
/quantum/train.py	P2	DDQN training loop + W&B logging
/quantum/retrain.py	P3	Offline re-training from Postgres replay table
/quantum/reward.py	P3	Reward engineering function (YAML-configurable weights)
/quantum/mock_data.py	P3	Synthetic learner archetype data generator
/quantum/visualise_circuit.py	P1	Renders VQC circuit diagram for report

/gen-engine/text_simplify.py	P3	LLaMA-3 text rewriting per reading level
/gen-engine/tts.py	P3	Coqui TTS wrapper with calm preset
/gen-engine/image_gen.py	P3	Stable Diffusion visual generation
/infra/docker-compose.yml	P3	Full stack: Postgres + Redis + Backend + Frontend

Important Config Constants (shared_config.py)

```
# shared_config.py - import this everywhere to stay in sync
STATE_VECTOR_DIM = 5          # [dwell, jitter, focus, stall, pref_delta]
ACTION_SPACE = 6              # 0=hold, 1=nudge, 2=simplify, 3=video, 4=game, 5=break
N_QUBITS = 5                  # must equal STATE_VECTOR_DIM
GAMMA = 0.99                  # RL discount factor
EPSILON_START = 1.0
EPSILON_END = 0.05
EPSILON_DECAY_EP = 500
REPLAY_CAPACITY = 10_000
BATCH_SIZE = 32
TARGET_UPDATE_FREQ = 100      # steps between soft target net updates
CONFIDENCE_GATE = 0.60        # min Q-value confidence to trigger gen engine
TELEMETRY_INTERVAL = 30_000   # ms - Observer polling interval
```

Tip: Keep shared_config.py at repo root and import it in both the Python backend and the quantum module. The frontend reads the same constants from a next.config.js env block to avoid magic numbers diverging across codebases.